

Converting a Python Notebook into a Streamlit App

Overview

This guide walks through converting a working Python notebook or script into an interactive Streamlit application.

You are encouraged to use AI tools (ChatGPT, Claude, Gemini, etc.) to help with parts of the conversion process, such as:

- turning notebook code into an `app.py` file
- creating Streamlit inputs and outputs
- improving layout and usability
- debugging errors
- explaining unfamiliar Streamlit functions
- organizing code into reusable functions

The goal is not to manually rewrite every line of code. The goal is to understand, test, refine, and improve the final application.

Using AI to Help Convert Your Code

Start with code that already works in a notebook or Python script. Then ask an AI tool to help convert it into a Streamlit app.

Example prompt:

I have a working Python notebook/script and want to convert it into a Streamlit app. Here is my code:

[paste code]

Please help me turn this into a clean Streamlit app. The app should allow users to interact with the workflow, adjust relevant inputs, and view the results. Please explain the generated code so I understand how it works.

You can make the prompt more specific by asking the AI tool to:

- add user inputs with `st.text_input`, `st.selectbox`, `st.slider`, or `st.number_input`
- display results with `st.dataframe`, `st.write`, `st.markdown`, or `st.metric`
- use `st.sidebar` for settings
- use `st.spinner` for longer operations
- use `st.session_state` when values need to persist across interactions
- use `@st.cache_data` or `@st.cache_resource` for slow data/model steps
- separate the code into multiple `.py` files, if appropriate

AI-generated code is a starting point. You are responsible for checking that it runs, understanding the logic, and fixing or improving the result.

Installing and Running Streamlit

If Streamlit is not already installed, install it from the command line:

```
pip install streamlit
```

Save your app as a Python file, commonly named:

```
app.py
```

Run the app from the command line in the same folder as your app file:

```
streamlit run app.py
```

Streamlit will open the app in your browser. If it does not open automatically, copy the local URL shown in the terminal and paste it into your browser.

Run from the folder that contains your data. Launch the app from the same folder as the files it reads (for example, your CSVs) — or point to them with a correct path. A common surprise is that code which worked in your notebook throws a `FileNotFoundError` in the app simply because the app was launched from a different folder.

How Streamlit Runs Your App

Streamlit re-runs your **entire script from top to bottom every time the user interacts with a widget** (a click, a slider move, a new text entry). This single idea explains two of the tools below:

- Wrap slow steps (loading data, fitting a model) in `@st.cache_data` / `@st.cache_resource` so they don't repeat on every interaction.
 - Use `st.session_state` to keep values you don't want to lose between re-runs.
-

Testing Your App

Test the app from the command line and in the browser.

Basic checks:

- Run the app with `streamlit run app.py`.
- Confirm the app opens without errors.
- Try different inputs and settings.
- Check that outputs update correctly.
- Restart the app and confirm it still works.
- Read any error messages in the terminal carefully.

If you change the code while the app is running, Streamlit usually detects the change and offers to rerun the app. If the app gets stuck, stop it in the terminal with:

```
Ctrl + C
```

Then restart it:

```
streamlit run app.py
```

Common Streamlit Components

User Input

Useful widgets include:

```
st.text_input()  
st.text_area()  
st.selectbox()  
st.slider()  
st.number_input()  
st.button()
```

Displaying Results

Useful output components include:

```
st.dataframe()  
st.write()  
st.markdown()  
st.expander()  
st.metric()
```

Sidebar

Use `st.sidebar` for controls and settings so the main page stays focused on results.

Example:

```
with st.sidebar:  
    st.header("Settings")
```

Session State

Use `st.session_state` to persist information across interactions when needed.

Example:

```
if "results" not in st.session_state:  
    st.session_state["results"] = []
```

Caching

Loading data or fitting models can be slow. Use caching to avoid unnecessary recomputation:

```
@st.cache_data  
@st.cache_resource
```

Tips for a Clean App

- Keep the interface simple and readable.
- Validate user inputs when appropriate.
- Use loading indicators during longer operations.
- Focus on making the workflow easy to follow.
- Test the app with multiple inputs and settings.
- Do not submit code that runs only once by accident or only works for one input.
- If your app calls an API, read the key from the environment or prompt for it — never hard-code it in the file you submit.

You may keep your app in a single `app.py` file or separate components into multiple modules (e.g., modeling logic, utility functions, UI). Modular organization is encouraged as a software engineering best practice but is not required.